



Web Development Technologies

Tutorial and Lab Sheet 1 Solution Guide

Exercise 1 to 3

These do not require any solutions, please follow instructions from the tutorial / lab sheet.

Exercise 4 Object Orientation in C#

(4.1) Start with this

```
using System;

class Building {
    public int floors;    // number of floors
    public int area;      // total square footage of building
    public int occupants; // number of occupants

    // Display the area per person.
    public void areaPerPerson() {
        Console.WriteLine("    " + area / occupants +
                           " area per person");
    }
}

// Use the areaPerPerson() method.
class BuildingDemo {
    public static void Main() {
        Building house = new Building();
        Building office = new Building();

        // assign values to fields in house
        house.occupants = 4;
        house.area = 2500;
        house.floors = 2;

        // assign values to fields in office
        office.occupants = 25;
        office.area = 4200;
        office.floors = 3;
    }
}
```



```

        Console.WriteLine("house has:\n " +
                           house.floors + " floors\n " +
                           house.occupants + " occupants\n " +
                           house.area + " total area");
        house.areaPerPerson();

        Console.WriteLine();

        Console.WriteLine("office has:\n " +
                           office.floors + " floors\n " +
                           office.occupants + " occupants\n " +
                           office.area + " total area");
        office.areaPerPerson();
    }
}

```

(4.2) Modified solution can be found in file **Building4.2.cs** - it is present in the code archive for this week's tutorial

(4.3) Read below:

Just add this in the declaration of **Building** class:

```

public Building(int f, int a, int o) {
    floors = f;
    area = a;
    occupants = o;
}

```

Accordingly you have to change the code in `Main()` method when you create an object.

You are advised to look for file **Building4.3.cs** in the code archive.

Exercise 5 Structs vs Classes in C#

A struct type is a value type that can contain constructors, constants, fields, methods, properties, indexers, operators, events, and nested types. The declaration of a struct takes the following form:

```
[attributes] [modifiers] struct identifier [:interfaces] body [;]
```

where:

attributes (Optional)

Additional declarative information. For more information on attributes and attribute classes, see 17. Attributes.

modifiers (Optional)

The allowed modifiers are `new` and the four access modifiers.

identifier

The struct name.

interfaces (Optional)

A list that contains the interfaces implemented by the struct, all separated by commas.

body

The struct body that contains member declarations.

The struct type is suitable for representing lightweight objects such as `Point`, `Rectangle`, and `Color`. Although it is possible to represent a point as a class, a struct is more efficient in some scenarios. For example, if you declare an array of 1000 `Point` objects, you will allocate additional memory for referencing each object. In this case, the struct is less expensive.

It is an error to declare a default (parameter less) constructor for a struct. A default constructor is always provided to initialize the struct members to their default values.

It is an error to initialize an instance field in a struct.

When you create a struct object using the new operator, it gets created and the appropriate constructor is called. Unlike classes, structs can be instantiated without using the new operator. If you do not use new, the fields will remain unassigned and the object cannot be used until all of the fields are initialized.

There is no inheritance for structs as there is for classes. A struct cannot inherit from another struct or class, and it cannot be the base of a class. Structs, however, inherit from the base class Object. A struct can implement interfaces, and it does that exactly as classes do.

This example declares a struct with three members: a property, a method, and a private field. It creates an instance of the struct and puts it to use:

```
// struct1.cs

using System;

struct SimpleStruct
{
    private int xval;

    public int X
    {
        get
        {
            return xval;
        }

        set
        {
            if (value < 100)
            {
                xval = value;
            }
        }
    }

    public void DisplayX()
    {
        Console.WriteLine("The stored value is: {0}", xval);
    }
}
```

```

class TestClass
{
    public static void Main()
    {
        SimpleStruct ss = new SimpleStruct();

        ss.X = 5;

        ss.DisplayX();
    }
}

```

Output

The stored value is: 5

Structs vs. Classes

Structs may seem similar to classes, but there are important differences that you should be aware of. First of all, classes are reference types and structs are value types. By using structs, you can create objects that behave like the built-in types and enjoy their benefits as well.

Heap or Stack?

When you call the New operator on a class, it will be allocated on the heap. However, when you instantiate a struct, it gets created on the stack. This will yield performance gains. Also, you will not be dealing with references to an instance of a struct as you would with classes. You will be working directly with the struct instance. Because of this, when passing a struct to a method, it's passed by value instead of as a reference.

And now the code:

It is a very simple code, which declares a struct that stores the coordinate information of a point and has a method to print those coordinates. Class Class1 makes use of the struct Point and swaps the coordinate –the important point is the line

```
static void Swap(ref Point p)
```

ref is used to pass the object 'p' of type Point as a reference-why do we that-why don't we pass it by value? Can you figure out this puzzle?

When **ref** modifies a reference parameter, it causes the reference to itself, to be passed by reference. This allows the method to change the object to which the reference refers.

Look for file **Point.cs** in this week's tutorial code archive.